

Universidade do Minho
Escola de Engenharia

Computação Gráfica

Trabalho Prático: Fase 3

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 03

A106919 Eduardo Freitas Fernandes
A106842 Gonçalo Rodrigues Ribeiro
A106888 José Mário Raimundo Lima

CG

Índice

1. Introdução	2
2. Organização do Código	2
3. Arquitetura do Sistema	2
4. <i>Bezier Patches</i>	3
5. <i>Vertex Buffer Objects</i>	3
5.1. <i>Index Buffer Objects</i>	3
6. Animações	4
6.1. Translação	4
6.2. Rotação	4
6.3. Integração no Grafo de Cena	5
7. <i>Demo Scene</i>	5
8. Conclusão	6

1. Introdução

Esta fase estende o sistema desenvolvido nas fases anteriores com três novas funcionalidades: geração de superfícies por *Bezier patches*, migração do *rendering* para *Vertex Buffer Objects* (VBOs) com *Index Buffer Objects* (IBOs), e suporte a animações temporais, translação ao longo de curvas de *Catmull-Rom* e rotação contínua.

A estrutura geral do sistema mantém-se: o *SceneParser* carrega o XML, a hierarquia de *Group* organiza a cena, e o *Engine* renderiza *frame a frame*. As alterações concentram-se na classe *Model* (que passa a usar VBOs), na adição de uma nova hierarquia *Animation*, e num novo modo no *Generator* para *patches*.

2. Organização do Código

Nesta terceira fase de desenvolvimento, a estrutura do projeto foi submetida a uma reorganização modular significativa, visando aumentar a escalabilidade e a separação de responsabilidades. Diferente da fase anterior, em que os ficheiros estavam agrupados em pastas genéricas (*src* e *include*), optou-se agora por uma distribuição por *packages* que refletem as camadas lógicas da aplicação:

- *engine/*: Concentra a lógica de *rendering* da cena, *parsing* de ficheiros XML e o ciclo principal.
- *generator/*: Contém as ferramentas de geração de primitivas geométricas e formas.
- *lib/*: Subdividida em *core/*, que contém as estruturas fundamentais de geometria (*Model*, *Point*, *Vector*), e *xml/*, que isola a dependência externa *tinyclml2*.

3. Arquitetura do Sistema

O diagrama de classes da Figura 1 reflete a maturidade da arquitetura, mantendo a consistência com as fases anteriores, mas introduzindo outros mecanismos. A estrutura permanece centrada na classe *World*, que atua como contentor raiz para a janela, câmara e a hierarquia de grupos de objetos.

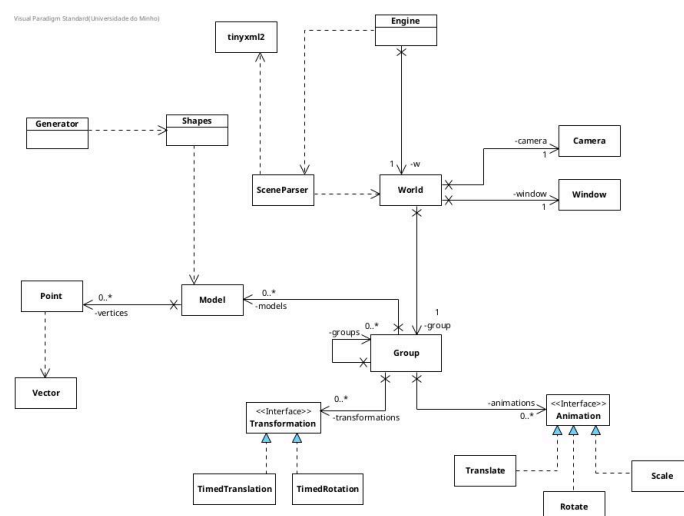


Figura 1: Arquitetura do Sistema

A principal evolução arquitetural desde a segunda fase reside na introdução da hierarquia *Animation*. Enquanto que na fase anterior as transformações geométricas eram estáticas, a arquitetura agora suporta transformações dependentes do tempo. Através da interface *Animation* e das suas especializações (*TimedTranslation* e *TimedRotation*), o motor é capaz de interpolar estados e atualizar a matriz de transformação de cada *Group* de forma cíclica.

As demais componentes, como a representação de vértices e vetores na camada *core* e a geração de malhas via *Generator*, mantêm-se como a base sólida sobre a qual estas novas funcionalidades de animação foram construídas.

4. Bezier Patches

Uma superfície *Bezier* cúbica é definida por uma grelha de $4 \times 4 = 16$ pontos de controlo. Dado um par de parâmetros $(u, v) \in [0, 1]^2$, o ponto na superfície calcula-se como:

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{i(u)} \cdot B_{j(v)} \cdot P_{ij} \quad (1)$$

onde B_i são os polinómios de *Bernstein* de grau 3:

$$B_0(t) = (1 - t)^3, \quad B_1(t) = 3t(1 - t)^2, \quad B_2(t) = 3t^2(1 - t), \quad B_3(t) = t^3 \quad (2)$$

O *Generator* recebe um ficheiro `.patch` com a topologia (índices dos 16 pontos de controlo por *patch*) e as coordenadas dos pontos de controlo. Para cada *patch*, divide o domínio (u, v) em `tessellation` $\times$ `tessellation` células e avalia $P(u, v)$ nos quatro cantos de cada célula, gerando dois triângulos por célula.

A invocação no *Generator* é:

```
./build/bin/generator patch <ficheiro.patch> <tessellation> <output.3d>
```

5. Vertex Buffer Objects

Nas fases anteriores, os vértices eram enviados ao OpenGL triângulo a triângulo via `glBegin/glEnd` (*immediate mode*). Nesta fase, os dados são carregados para memória da GPU numa única operação e reutilizados em cada *frame*.

A classe *Model* passa a manter dois *buffers* na GPU:

- **VBO** (*Vertex Buffer Object*): armazena as coordenadas únicas dos vértices como `float`.
- **IBO** (*Index Buffer Object*): armazena, para cada triângulo, os índices dos três vértices no VBO.

O método `setup()` é chamado uma vez após o carregamento da cena e envia os dados para a GPU via `glBufferData`. O método `draw()` limita-se a ligar os *buffers* e emitir um único `glDrawElements`, sem transferir dados entre CPU e GPU.

5.1. Index Buffer Objects

A função `generateBuffers` (em `utils.cpp`) constrói o VBO e o IBO a partir da lista de vértices do modelo. Usa um `std::map<Point, unsigned int>` para detetar pontos duplicados: a primeira ocorrência de um ponto é adicionada ao VBO e recebe um índice novo; ocorrências

repetidas reutilizam o índice existente. O resultado é um VBO compacto (apenas vértices únicos) e um IBO que descreve a triangulação por referência a esses vértices.

6. Animações

As transformações geométricas estáticas foram expandidas para suportar comportamentos dinâmicos baseados no tempo real. O tempo decorrido, obtido através da função `glutGet(GLUT_ELAPSED_TIME)`, é convertido para segundos e propagado através do grafo de cena para atualizar as transformações dinâmicas.

6.1. Translação

A translação dinâmica permite que um modelo percorra uma trajetória definida por um conjunto de pontos de controlo que formam uma curva de *Catmull-Rom*. Esta curva garante continuidade C^1 , passando por todos os pontos de controlo intermédios, exceto o primeiro e o último.

O cálculo da posição $P(t)$ num segmento da curva, onde $t \in [0, 1]$, é realizado através da seguinte equação matricial:

$$P(t) = \frac{1}{2} \begin{pmatrix} t^3 & t^2 & t & 1 \end{pmatrix} \begin{pmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} P_{i-1} \\ P_i \\ P_{i+1} \\ P_{i+2} \end{pmatrix} \quad (3)$$

Quando o atributo `align` é ativado, o modelo é orientado segundo o vetor tangente à curva, $P'(t)$, calculado pela derivada da posição em ordem a t :

$$P'(t) = \frac{1}{2} \begin{pmatrix} 3t^2 & 2t & 1 & 0 \end{pmatrix} \begin{pmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} P_{i-1} \\ P_i \\ P_{i+1} \\ P_{i+2} \end{pmatrix} \quad (4)$$

A orientação é completada através do cálculo de uma base ortonormada ($X_{\text{local}}, Y_{\text{local}}, Z_{\text{local}}$) recorrendo a produtos vetoriais entre a tangente (eixo X) e um vetor auxiliar de verticalidade $(0, 1, 0)$.

A lógica de translação foi encapsulada na classe `TimedTranslation`. Esta entidade armazena os pontos de controlo e o período temporal, delegando o cálculo da curva para funções utilitárias que implementam a multiplicação matricial de *Catmull-Rom*. O alinhamento é resolvido dinamicamente: a tangente calculada é normalizada para definir o eixo X local, e os restantes eixos são deduzidos por produtos vetoriais, resultando numa matriz de rotação 4×4 injetada diretamente no *pipeline* gráfico.

6.2. Rotação

A rotação temporal define a velocidade de rotação de um objeto em torno de um eixo, especificada pelo tempo necessário para completar uma volta completa. O ângulo de rotação α num instante Δt é calculado proporcionalmente ao ciclo definido:

$$\alpha = \left(\frac{\Delta t}{\text{tempo}} \right) \times 360^\circ \quad (5)$$

Esta funcionalidade é gerida pela classe `TimedRotation`, que converte o tempo absoluto da aplicação num ângulo acumulado para a função de rotação do *OpenGL*. Esta abordagem garante que o movimento seja fluido e independente da taxa de atualização (*frame rate*) da aplicação.

6.3. Integração no Grafo de Cena

A ligação entre o tempo global e as transformações individuais é feita através da classe `Group`. Durante a renderização, cada grupo executa uma fase de atualização de animações antes de desenhar os seus modelos:

1. O tempo atual é capturado e passado às instâncias de animação.
2. Cada animação (`TimedRotation` ou `TimedTranslation`) calcula o seu estado atual e aplica as transformações à matriz de visualização.
3. Como estas operações ocorrem dentro do contexto de matriz do grupo, todos os subgrupos e modelos descendentes herdam automaticamente o movimento animado, respeitando a hierarquia da cena.

7. Demo Scene

Como prova de conceito das funcionalidades implementadas, foi desenvolvida uma cena representativa do Sistema Solar, focada na aplicação prática de hierarquias de transformação e animações temporais.

- **Hierarquia de Transformações:** A cena utiliza uma estrutura de grafos de cena onde os planetas e satélites são definidos como grupos dependentes. Esta organização permite que as luas herdem as transformações de translação e rotação dos seus respetivos grupos, mantendo as suas próprias órbitas.
- **Dinâmica Orbital:** As órbitas foram modeladas através da aplicação de rotações baseadas no tempo, permitindo velocidades angulares distintas para cada corpo celeste, simulando a mecânica orbital de forma contínua.
- **Geometria Complexa e Curvas de Bézier:** A cena inclui um bule, utilizado para demonstrar a capacidade do motor em processar modelos gerados a partir de **Bézier Patches**. Este modelo foi integrado numa trajetória 3D dinâmica utilizando uma curva de Catmull-Rom, com variação nos três eixos cartesianos e alinhamento automático.

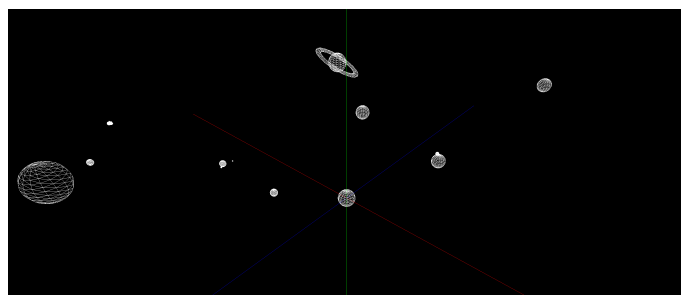


Figura 2: *Demo Scene*

8. Conclusão

Esta fase introduziu três extensões ao sistema. Os VBOs eliminam a transferência de dados por *frame*, delegando ao OpenGL a gestão eficiente da geometria na GPU. Os *Bézier patches* permitem gerar superfícies curvas suaves a partir de ficheiros de controlo padrão, como o *teapot*. As animações temporais, curva de Catmull-Rom e rotação contínua, tornam a cena dinâmica, com objetos a orbitar e a rodar de forma autónoma e configurável pelo XML.